MobaLiveCD Rust/Tauri Conversion Plan

🚀 Vision: Modern Cross-Platform Application

Convert the current Python/GTK4 MobaLiveCD application to a modern Rust/Tauri/React stack for better performance, reliability, and user experience.

📋 Current Issues This Would Solve

GTK/Python Problems

- ❌ GTK4/Adwaita widget management issues (AdwPreferencesGroup removal crashes)
- ❌ Python dependency hell (psutil, gi, gobject-introspection)
- ❌ Complex GTK UI building and debugging
- ❌ Platform-specific GTK theming and layout issues
- ❌ Memory leaks in long-running GTK applications

Benefits of Rust/Tauri/React

- ✅ **Memory Safety**: Rust eliminates segfaults and memory leaks
- ✅ **Modern UI**: React with CSS/Tailwind instead of GTK theming
- ✅ **Single Binary**: No Python runtime or GTK dependencies needed
- ✅ **Cross-Platform**: Windows, macOS, Linux support out of the box
- ✅ **Better Performance**: Native Rust backend with web-based frontend
- ✅ **Developer Experience**: Hot reload, modern debugging tools
- ✅ **Maintainability**: TypeScript types and modern tooling

🗺️ Implementation Roadmap

Phase 1: Project Setup

- [ ] Initialize Tauri project with React frontend
- [ ] Set up TypeScript, Tailwind CSS, and build tools
- [ ] Configure Tauri permissions and security settings
- [ ] Set up CI/CD pipeline for multi-platform builds

Phase 2: Core Backend (Rust)

- [ ] **NVMe Detection Module**
  - Port `nvme_handler.py` to Rust using `sysfs` and `lsblk` parsing
  - Use `serde` for JSON serialization
  - Implement proper error handling with `anyhow` or `thiserror`
- [ ] **QEMU Runner Module**
  - Port `qemu_runner.py` to Rust using `std::process`
  - Implement AI-optimized profile detection
  - Handle process management and monitoring
- [ ] **USB Creator Module**
  - Port USB creation functionality to Rust
  - Use `dd` or native block device operations
  - Add progress tracking and cancellation support

Phase 3: Tauri Commands API

```
#[tauri::command]
async fn get_nvme_devices() -> Result<Vec<NVMeDevice>, String>

#[tauri::command]
async fn get_partitions(device: String) -> Result<Vec<Partition>, String>

#[tauri::command]
async fn run_qemu(boot_source: String, options: QEMUOptions) -> Result<(),
String>

#[tauri::command]
async fn create_usb(iso_path: String, usb_device: String) -> Result<(), String>
```

Phase 4: React Frontend

- [ ] **Main Application Layout**
  - Clean, modern design with Tailwind CSS
  - Dark/light theme support
  - Responsive layout for different screen sizes

- [ ] **Partition Selection Component**
  - Tree view of NVMe devices and partitions
  - Visual indicators for bootable/ZFS/mounted partitions
  - Real-time refresh capabilities
- [ ] **QEMU Configuration Panel**
  - AI-detected OS profiles with custom options
  - Resource allocation sliders (RAM, CPU cores)
  - Advanced QEMU options for power users
- [ ] **USB Creation Wizard**
  - Step-by-step USB creation process
  - Progress tracking with cancellation
  - Device safety warnings and confirmations

Phase 5: Advanced Features

- [ ] **System Integration**
  - File associations for ISO files
  - Desktop integration and notifications
  - Auto-privilege elevation when needed
- [ ] **Enhanced Functionality**
  - QEMU snapshot management
  - Network boot (PXE) support
  - Virtual machine templates and presets
  - Logging and diagnostics panel

Phase 6: Testing & Distribution

- [ ] **Testing Framework**
  - Unit tests for Rust backend modules
  - Integration tests for Tauri commands
  - End-to-end tests for critical workflows
- [ ] **Packaging & Distribution**
  - AppImage for Linux

- MSI installer for Windows
- DMG bundle for macOS
- AUR package for Arch Linux
- Flatpak for universal Linux distribution

🛠️ Technical Architecture

Backend (Rust)

```
src/
├── main.rs              # Tauri main entry point
├── commands/            # Tauri command handlers
│   ├── nvme.rs          # NVMe detection and management
│   ├── qemu.rs          # QEMU runner and process management
│   └── usb.rs           # USB creation utilities
├── core/                # Core business logic
│   ├── device.rs        # Device detection and validation
│   ├── filesystem.rs    # Filesystem type detection
│   └── process.rs       # Process management utilities
└── utils/               # Shared utilities
    ├── error.rs         # Error handling types
    └── config.rs        # Configuration management
```

Frontend (React/TypeScript)

```
src/
├── App.tsx              # Main application component
├── components/          # Reusable UI components
│   ├── DeviceTree.tsx   # NVMe device/partition tree
│   ├── QEMUConfig.tsx   # QEMU configuration panel
│   └── USBWizard.tsx    # USB creation wizard
├── hooks/               # Custom React hooks
│   ├── useDevices.ts    # Device detection state
│   └── useQEMU.ts       # QEMU process management
├── types/               # TypeScript type definitions
│   └── api.ts           # API response types
└── utils/               # Frontend utilities
    └── formatting.ts    # Data formatting helpers
```

🎯 Migration Strategy

Gradual Migration Approach

1. **Start with backend modules**: Port core functionality to Rust
2. **Create parallel Tauri app**: Build alongside existing Python app
3. **Feature parity testing**: Ensure all functionality works
4. **User testing**: Beta testing with existing users
5. **Full migration**: Replace Python version

Data Compatibility

- Maintain configuration file compatibility
- Preserve user preferences and settings
- Support existing QEMU profiles and templates

🚧 Development Environment Setup

Prerequisites

```
# Install Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Install Node.js and npm
sudo pacman -S nodejs npm  # Arch/Garuda Linux

# Install Tauri CLI
cargo install tauri-cli

# Install frontend dependencies
npm install -g @tauri-apps/cli
```

Development Workflow

```
# Start development server with hot reload
npm run tauri dev

# Build for production
npm run tauri build

# Run tests
cargo test
npm test
```

📊 Expected Benefits

Performance Improvements

- **Startup time**: 3-5x faster than Python/GTK
- **Memory usage**: 50-70% reduction in RAM usage
- **Binary size**: Single ~15MB executable vs 100MB+ Python environment

User Experience

- **Modern UI**: Clean, responsive interface
- **Better error handling**: User-friendly error messages
- **Consistent behavior**: Same experience across all platforms
- **Auto-updates**: Built-in update mechanism

Developer Experience

- **Type safety**: Rust + TypeScript prevent runtime errors
- **Better testing**: Comprehensive test coverage
- **Modern tooling**: IDE support, debugging, profiling
- **Community**: Active Rust/Tauri/React ecosystems

🎯 Success Metrics

- [ ] Feature parity with current Python version
- [ ] 90%+ test coverage for critical functionality
- [ ] Sub-3-second application startup time
- [ ] Cross-platform builds working on Windows/macOS/Linux
- [ ] Positive user feedback from beta testing
- [ ] Zero critical bugs in production release

This conversion plan represents a modern approach to rebuilding MobaLiveCD with industry-standard tools and practices. The investment in migration will pay dividends in maintainability, performance, and user satisfaction.